
How the XAI Hyper-Heuristic System Works

A plain-English guide to the xai-sphh project
from beginner to technical detail

The Big Question

When a computer program is automatically solving hard problems — scheduling, routing, planning — **what is it actually learning to pay attention to?** And does it pay attention to the same things no matter which problem it is solving?

Contents

1	The Big Picture (30-Second Version)	2
2	The Two Layers of the System	2
3	The Problem Domains — What Are We Actually Solving?	2
4	How the Hyper-Heuristic Works (High Level)	3
5	Selection Strategies — Picking the Right Tool	4
5.1	Simple Random (SR) — The Baseline	4
5.2	Choice Function (CF) — The Learner	4
6	Acceptance Strategies — Keeping or Discarding Results	4
7	Single-Point vs Multi-Point Search	4
8	What Gets Recorded — The Trace File	5
9	The Python Pipeline — From Data to Explanation	5
10	The Explainability Layer — SHAP and LIME	6
10.1	SHAP — Global Explanation	6
10.2	LIME — Local Explanation	7
11	The Surrogate Model — How Faithful Is It?	7
12	How Datasets Feed Into the System	7
12.1	The instanceIndex Field	7
12.2	Running the Full Experimental Matrix	8
13	The Entire System in One Diagram	8
14	Summary	9

The Big Picture (30-Second Version)

Imagine you are trying to solve a puzzle, but you have ten different tools available. You do not know in advance which tool is best — you have to try them and learn as you go.

A **hyper-heuristic** is a program that does exactly this. It sits above a set of simple problem-solving tools (called **low-level heuristics**) and decides at every step:

1. **Which tool should I use right now?**
2. **Should I keep the result it produced, or throw it away?**

Our project records every single one of these decisions — hundreds of thousands of them — and then uses **Explainable AI** to ask: *what was the program paying attention to when it made each decision?*

Why does this matter? Most AI decision-makers are black boxes. You get an answer, but you do not know why. XAI opens the box and shows you the reasoning, building trust and revealing what actually works.

The Two Layers of the System

The whole system splits into two parts that run one after the other.



- **Part 1 (Java)** runs a hyper-heuristic on real benchmark problems and writes a detailed log of every decision made — which tool was used, what the result was, and whether it was accepted.
- **Part 2 (Python)** reads those logs, trains an AI model to replicate the decision pattern, and then explains the AI model using SHAP and LIME.

The Problem Domains — What Are We Actually Solving?

The system does not run on made-up data. It runs on **six real-world benchmark problems** from the CHeSC 2011 international competition. These are standard test problems used by researchers worldwide.

The data for all six problems is already bundled inside the HyFlex jar files (the library files in `lib/hyflex/`). You do not import external datasets — the instances are pre-loaded.

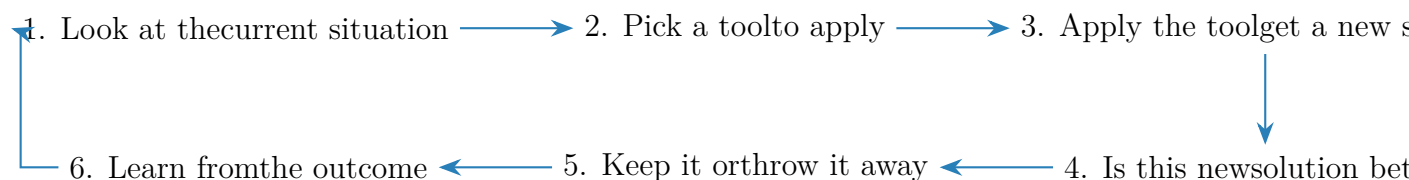
Domain	What problem is being solved?	What does the score measure?
SAT	Find variable assignments (true/false) that satisfy a Boolean formula with thousands of clauses.	Number of unsatisfied clauses. <i>Goal: reach 0.</i>
Bin Packing	Pack items of varying sizes into the fewest possible bins.	Number of bins used. <i>Goal: minimise.</i>
Flow Shop	Schedule a set of jobs across multiple machines to finish as quickly as possible.	Total time to finish all jobs (makespan). <i>Goal: minimise.</i>
TSP (Travelling Salesman)	Find the shortest route that visits every city exactly once and returns home.	Total route distance. <i>Goal: minimise.</i>
VRP (Vehicle Routing)	Route a fleet of vehicles to serve all customers within capacity and time limits.	Total travel distance. <i>Goal: minimise.</i>
Personnel Scheduling	Assign staff to shifts satisfying skills, availability, and fairness rules.	Penalty points for broken rules. <i>Goal: reach 0.</i>

Each domain has **10 pre-loaded instances** (indexed 0–9), ranging from easy to very hard. You select which one to use in the config file using the `instanceIndex` field.

Key insight: The hyper-heuristic *never sees the actual problem data* — no cities, no jobs, no clauses. All it ever receives is a single number: the objective score. This is what makes it a *cross-domain* solver. The exact same algorithm works on all six problems without any changes.

How the Hyper-Heuristic Works (High Level)

Think of the hyper-heuristic as a student sitting an exam with a toolbox.



This loop runs **100,000 times** per experiment. Each pass is called an **iteration**. At every iteration, both the selection decision (step 2) and the acceptance decision (step 5) are recorded in the trace file.

Selection Strategies — Picking the Right Tool

There are two ways the hyper-heuristic decides which low-level heuristic to apply next.

Simple Random (SR) — The Baseline

Picks a tool completely at random. No memory, no learning. This is the **baseline** — we use it to compare against smarter strategies.

Choice Function (CF) — The Learner

Keeps a score for every tool based on three things:

Term	Name	What it measures
f1	Individual performance	How well has this tool done on its own in the past?
f2	Pair performance	How well does this tool do <i>after</i> the tool used last time?
f3	Time since last used	How long ago was this tool used? (promotes variety)

The final score is: $CF(h) = \varphi \cdot f1 + \varphi \cdot f2 + (1 - \varphi) \cdot f3$

The weight φ adapts automatically — it jumps up when the search is improving and slowly decays when it is not, so the system shifts between exploiting good tools and exploring idle ones.

Acceptance Strategies — Keeping or Discarding Results

After applying a tool, the system must decide: *do we keep this new solution or stick with the old one?*

Strategy	Rule
Late Acceptance (LA)	Accept if the new solution is at least as good as the solution from L steps ago. Allows temporary worsening to escape local optima.
AILTA	Adaptive version of LA. The threshold decays over time, becoming stricter as the budget runs out.
Simulated Annealing (SA)	Sometimes accepts worse solutions based on probability. The chance of accepting bad solutions decreases over time, like metal cooling.
Improving or Equal	Only keep if the new solution is at least as good. Simple and strict.
Naive	Always keep. Used only for debugging.

Single-Point vs Multi-Point Search

The system supports two fundamentally different search modes.

Single-Point (SP)

Works with **one solution**
at a time.

Tries to improve it step by step.

Needs an acceptance strategy to
decide whether to
keep each step.

Multi-Point (MP)

Keeps a **population** of
solutions (e.g. 10 at once).

Can combine two solutions
using crossover heuristics.

Replaces the worst individual
if a better one is found.

For multi-point search, the replacement rule acts as acceptance — there is no separate acceptance strategy. An elitism guard ensures the best solution in the population is never overwritten.

What Gets Recorded — The Trace File

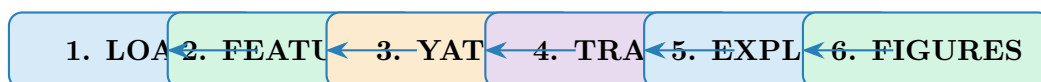
Every single iteration is written to a CSV file called a **trace**. Each row captures a complete snapshot of one decision:

Column	What it means
run_id	Unique ID for this experiment run
iteration	Which step number this is (0 to 99,999)
heuristic_id	Which low-level tool was selected
heuristic_class	What type of tool: MUTATION, LOCAL_SEARCH, etc.
objective_before	Score before the tool was applied
objective_after	Score after the tool was applied
delta	The change in score (negative = improvement)
accepted	Was this result kept? (true/false)
best_so_far	Best score seen at any point in the run
feat_progress	How far through the budget we are (0.0 to 1.0)
feat_iters_since_accept	How many steps since we last kept a solution
feat_improve_rate_50	Fraction of the last 50 steps that improved
feat_pop_diversity	How spread out the population is (MP only)
...	(16 features total)

A single 100,000-iteration run produces a trace file of approximately 20–42 MB. The current baseline dataset has **400,000 rows** (4 technique runs combined).

The Python Pipeline — From Data to Explanation

Once the Java side has produced trace files, the Python pipeline processes them in six sequential stages.



Stage 1 — Load

Reads all CSV trace files, checks they match the schema (all 19 required columns present, correct data types), and saves everything into a single efficient Parquet file. *Input: CSV files. Output: traces.parquet.*

Stage 2 — Features

Selects the 16 `feat_*` columns as inputs to the model (X) and the `accepted` column as what the model must predict (y). Missing population features for single-point runs are filled with `-1` (a sentinel meaning “not applicable”). *Output: X.parquet, y.parquet.*

Stage 3 — Yates Measures

Computes three statistical measures adapted from Yates et al.: (1) log returns per heuristic — which tools give the best improvements on average; (2) percentile ranks of objective values; (3) footrule distance — how much the ranking of tools changes between early and late search.

Stage 4 — Train

Trains a **Gradient Boosting** classifier on the 400,000 rows. The model learns to predict, given the state features, whether the hyper-heuristic will accept or reject. Performance is measured with 5-fold cross-validation.

Stage 5 — Explain

Applies SHAP and LIME to explain the surrogate model. SHAP gives a global picture (which features matter most across all decisions). LIME gives a local picture (why was this specific decision made?).

Stage 6 — Figures

Produces publication-ready PDF figures: the convergence curve (best score over time) and the SHAP importance bar chart.

The Explainability Layer — SHAP and LIME

SHAP — Global Explanation

SHAP (SHapley Additive exPlanations) answers: “**which features matter most, on average, across all 400,000 decisions?**”

It assigns each feature a score measuring how much it contributes to the prediction. A high SHAP value means the feature strongly influences whether the hyper-heuristic accepts or rejects.

Feature	Mean SHAP	What it means
<code>feat_pop_target_rank</code>	1.26	Where the target sits in the population
<code>feat_pop_diversity</code>	0.99	How spread out the population is
<code>feat_improve_rate_50</code>	0.75	Recent improvement frequency
<code>feat_iters_since_accept</code>	0.73	How long since the last accepted move
<code>feat_progress</code>	0.47	How far through the budget we are
<code>feat_mean_logret_50</code>	0.45	Average quality of recent moves
<code>feat_iters_since_improve</code>	0.45	How long since best score improved

What this tells us: The hyper-heuristic pays most attention to *where it is in the population, how diverse the population is, and how recently it made progress*. It is responding to search momentum and population structure — not to anything problem-specific.

LIME — Local Explanation

LIME (Local Interpretable Model-agnostic Explanations) answers: **“why was this specific decision made at iteration 50,000?”**

It creates a small perturbed dataset around a single decision and fits a simple linear model to explain just that one prediction. Useful for auditing individual decisions that seem surprising.

The Surrogate Model — How Faithful Is It?

Before trusting the SHAP and LIME explanations, we need to know whether the surrogate model actually represents the real hyper-heuristic behaviour well.

Metric	Surrogate	Trivial baseline
Accuracy (fraction correct)	88.5%	57.5%
ROC-AUC (ranking quality)	95.3%	50.0%
F1 Score (balanced accuracy)	90.5%	—

The trivial baseline just always predicts the most common class. Our surrogate is 31 percentage points better than that — it has genuinely learned the acceptance policy. The SHAP and LIME explanations are therefore valid descriptions of how the hyper-heuristic actually behaves.

How Datasets Feed Into the System

Short answer: You do not import your own datasets. The benchmark problem instances are already bundled inside the HyFlex jar files. You control which instance to run using a single number in the config file.

The `instanceIndex` Field

Each of the six problem domains has 10 pre-loaded instances (0–9), ranging from small and easy to large and hard. You select one using `instanceIndex` in the YAML config:

```

techniqueName: SP-CF-LA
domain: SAT
instanceIndex: 3    # pick instance 3 out of 10
seed: 42
budgetType: iterations
budgetValue: 100000
outputDirectory: data/raw/traces

```

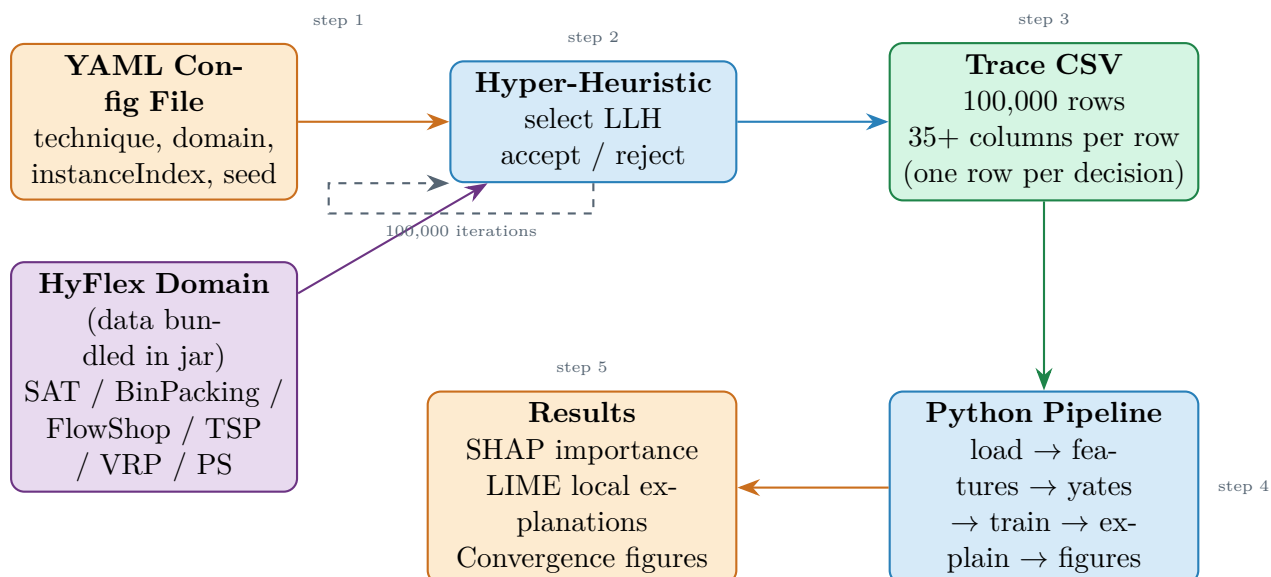
Running the Full Experimental Matrix

For a complete study you would run every combination of technique, domain, instance and seed. A typical matrix:

Dimension	Count	Values
Techniques	4	SP-SR-LA, SP-CF-LA, SP-CF-AILTA, MP-CF
Domains	6	SAT, BinPacking, FlowShop, TSP, VRP, PS
Instances	10	0 through 9
Seeds	5	42, 123, 456, 789, 1000
Total runs	1,200	

All 1,200 trace files drop into `data/raw/traces/`. The Python pipeline automatically picks up every CSV in that folder and combines them for analysis.

The Entire System in One Diagram



Summary

1. **The system solves real benchmark problems** from six domains (SAT, Bin-Packing, FlowShop, TSP, VRP, PersonnelScheduling) using a hyper-heuristic that selects and applies low-level tools automatically.
2. **The problem data is pre-bundled.** You select which instance to run using `instanceIndex`: 0--9 in the config file.
3. **The hyper-heuristic is blind to the problem.** It only ever sees an objective score — never cities, clauses, jobs, or routes. This is what makes it cross-domain.
4. **Every decision is recorded.** A 100,000-iteration run produces a trace file with 100,000 rows and 35+ columns.
5. **A surrogate AI model learns the decision pattern** (88.5% accuracy) from the recorded traces.
6. **SHAP reveals what the system pays attention to globally.** Population structure, search stagnation, and recent improvement rate dominate acceptance decisions — not the problem type.
7. **LIME explains individual decisions locally.** You can audit any specific iteration and see exactly what pushed the decision towards acceptance or rejection.

Generated from the xai-sphh project source code. All metrics reflect the 4-technique baseline experiment (SP-SR-LA, SP-CF-LA, SP-CF-AILTA, MP-CF) run on SAT instance 0, 100,000 iterations each.